

Algoritmos de Ordenação

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- **Ordenação** é pré-processamento em muitas aplicações
- Fácil comparar dois algoritmos quando aplicados a uma mesma massa de dados
- Entretanto, massas de dados modelam situações
 - Particularidades
 - Alguns métodos se comportam melhor que outros, independente de sua avaliação em testes genéricos

Problema de Ordenação

- Um vetor $v[1..n]$ é crescente se $v[1] \leq v[2] \leq \dots \leq v[n]$.
O problema da ordenação de um vetor consiste no seguinte:
 - Rearranjar (ou seja, permutar) os elementos de um vetor $v[1..n]$ de tal modo que ele se torne crescente

Ordenação por Bolha (*Bubble Sort*)

- Simples
- Mais conhecido
- **Ideia:** Uma iteração percorre a tabela do início até o fim, sem interrupção
 - Troca dois elementos consecutivos se estiverem fora de ordem
 - Ao término, maior elemento estará na última posição
 - Na segunda iteração, o segundo maior elemento é posicionado, e assim sucessivamente ...
 - Processamento é repetido $n - 1$ vezes

Ordenação por Bolha: Visualização Iteração

0	1	2	3	4	5	6	
40	37	95	42	39	51	60	Tabela inicial
40	37	95	42	39	51	60	$i = 0 \quad j = 0$
37	40	95	42	39	51	60	$i = 0 \quad j = 1$
37	40	95	42	39	51	60	$i = 0 \quad j = 2$
37	40	42	95	39	51	60	$i = 0 \quad j = 3$
37	40	42	39	95	51	60	$i = 0 \quad j = 4$
37	40	42	39	51	95	60	$i = 0 \quad j = 5$
37	40	42	39	51	60	95	$i = 0 \quad j = 6$

Passo a passo de uma iteração de ordenação por bolha.

Ordenação por Bolha: Exemplo

0	1	2	3	4	5	6	
40	37	95	42	39	51	60	Tabela inicial
37	40	42	39	51	60	95	após 1ª iteração (i = 0)
37	40	39	42	51	60	95	após 2ª iteração (i = 1)
37	39	40	42	51	60	95	após 3ª iteração (i = 2)
37	39	40	42	51	60	95	após 4ª iteração (i = 3)

Um exemplo de ordenação por bolha.

Ordenação por Bolha: Algoritmo

```
1. void bubble_sort(int v[], int n) {  
2.     int i, j, aux;  
3.  
4.     for (i = 0; i < n - 1; i++) {  
5.         for (j = 0; j < n - 1 - i; j++) {  
6.             if (v[j] > v[j + 1]) {  
7.                 aux = v[j];  
8.                 v[j] = v[j + 1];  
9.                 v[j + 1] = aux;  
10.            }  
11.        }  
12.    }  
13.}
```

Ordenação Por Bolha: Complexidade

- **Pior caso:** $O(n^2)$
- **Melhor caso:** $O(n^2)$
- Obs.: Complexidades são as mesmas devido aos percursos estipulados para as variáveis i e j
 - É possível alterar e melhorar o **melhor caso**?

Ordenação por Inserção (*Insertion Sort*)

- Simples
- Complexidade equivalente à ordenação bolha
- **Ideia:** Imagine uma tabela ordenada até o i -ésimo elemento
 - Ordenação pode ser estendida até o $(i + 1)$ -ésimo elemento por meio de comparações sucessivas deste $(i + 1)$ com os elementos anteriores,
 - isto é, com o i -ésimo elemento, com o $(i - 1)$ -ésimo elemento etc., procurando sua posição correta na parte da tabela que já está ordenada

Ordenação por Inserção: Visualização Iteração

0	1	2	3	4	5	6	
40	37	95	42	23	51	27	tabela inicial
40	37	95	42	23	51	27	$i = 1$ $j = 0$
37	40	95	42	23	51	27	

Passo a passo de uma iteração de ordenação por inserção.

Ordenação por Inserção: Exemplo

0	1	2	3	4	5	6	
40	37	95	42	23	51	27	Tabela inicial
37	40	95	42	23	51	27	Após $i = 1$
37	40	95	42	23	51	27	Após $i = 2$
37	40	42	95	23	51	27	Após $i = 3$
23	37	40	42	95	51	27	Após $i = 4$
23	37	40	42	51	95	27	Após $i = 5$
23	27	37	40	42	51	95	Após $i = 6$

Um exemplo de ordenação por inserção.

Ordenação por Inserção: Algoritmo

```
1. void insertion_sort(int v[], int n) {
2.     int i, chave, j;
3.     for (i = 1; i < n; i++) {
4.         chave = v[i];
5.         j = i - 1;
6.
7.         // Mover os elementos de v[0..i-1], que são maiores que chave,
8.         // uma posição à frente de sua posição atual
9.         while (j >= 0 && v[j] > chave) {
10.            v[j + 1] = v[j];
11.            j = j - 1;
12.        }
13.        v[j + 1] = chave;
14.    }
15.}
```

Ordenação por Inserção: Complexidade

- **Pior caso:** $O(n^2)$
- **Melhor caso:** $O(n)$
- Percurso em cada iteração termina exatamente quando a inversão não ocorre
- Então, o algoritmo tem complexidade de **melhor caso** $O(n)$, que ocorre quando o número de inversões é zero
- No **pior caso**, quando a tabela está em ordem inversa, são executadas $n - 1$ iterações (para $i = 2, \dots, n$) e, em cada uma delas, executadas $i - 1$ inversões. Logo

$$\sum_{i=2}^n Inv(i) = 1 + 2 + \dots + (n - 1) = O(n^2)$$

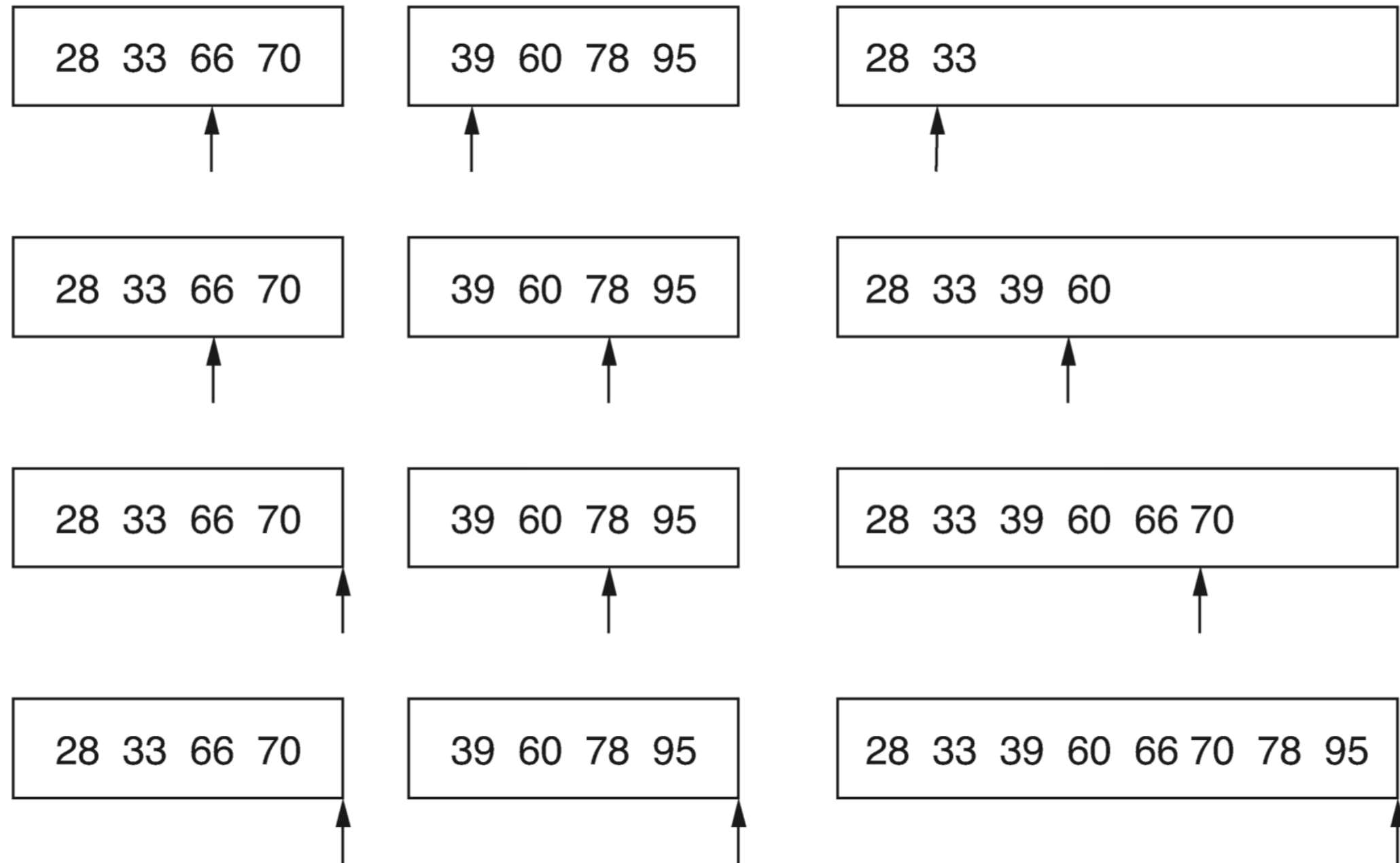
Ordenação por Comparação

- **Ordenação por comparação** usa apenas comparações entre elementos para ganhar informação de ordem sobre sequência de entrada $\{a_1, a_2, \dots, a_n\}$.
- Ordenação por bolha, ordenação por inserção, ordenação por intercalação, Quicksort são algoritmos de comparação por ordenação
- Pior caso é $\Omega(n \log n)$
 - Ou seja, o pior caso sempre irá ter complexidade assintótica limitada inferiormente por $n \log n$ (prova em “*Introduction to Algorithms*”, Cormen)

Ordenação por Intercalação (*Merge Sort*)

- Procedimento básico intercalação de listas
- **Ideia:** intercalar as duas metades da lista desejada quando estas já se encontram ordenadas
 - Na realidade, deseja-se então ordenar primeiramente as duas metades, o que pode ser feito utilizando recursivamente o mesmo conceito

Ordenação por Intercalação: Intercalação



Processo de intercalação.

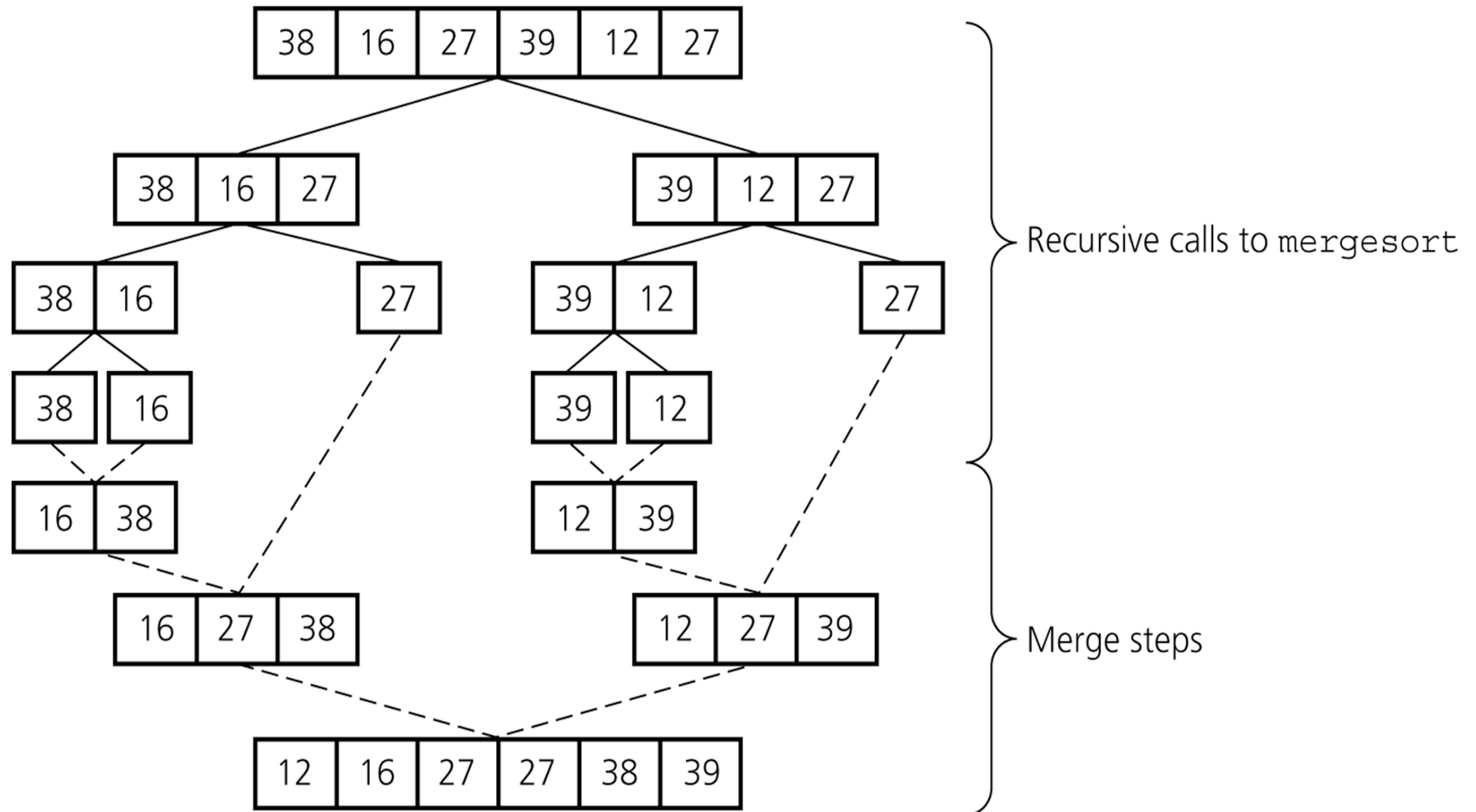
Ordenação por Intercalação: Método de Ordenação

- $\mathcal{L}$ uma lista que se deseja ordenar
- Divide-se $\mathcal{L}$ em duas metades e as ordena
 - Resultado são duas listas ordenadas que podem ser intercaladas
- Para ordenar cada uma das metades o processo considerado é o mesmo, sendo o problema dividido em problemas menores, que são sucessivamente solucionados

Ordenação por Intercalação: Algoritmo

```
1. // Função principal de ordenação usando o Merge Sort
2. void mergesort(int l[], int esq, int dir) {
3.     if (esq < dir) {
4.         int centro = (dir + esq) / 2;
5.
6.         // Ordena as duas metades
7.         mergesort(l, esq, centro);
8.         mergesort(l, centro + 1, dir);
9.
10.        // Intercala as duas metades ordenadas
11.        intercala(l, esq, centro, dir);
12.    }
13.}
```

Ordenação por Intercalação: Exemplo



Ordenação por Intercalação: Complexidade

- Complexidade de **pior caso** $O(n \log n)$
- Entretanto, mesmo com este bom resultado a ordenação por intercalação não é um dos métodos de ordenação mais empregados
 - Sua eficiência depende da cuidada implementação da tabela temporária

Quicksort

- Tradução literal seria “ordenação rápida”
- Um dos mais eficientes
- **Ideia:** Dada tabela $\mathcal{L}$ com n elementos, procedimento recursivo para ordenar $\mathcal{L}$:
 - se $n = 0$ ou $n = 1$ então a tabela está ordenada
 - escolha elemento x em $\mathcal{L}$ - este elemento é chamado **pivô**
 - separe $\mathcal{L} - \{x\}$ em dois conjuntos de elementos disjuntos:
 $S_1 = \{w \in \mathcal{L} - \{x\} \mid w < x\}$ e
 $S_2 = \{w \in \mathcal{L} - \{x\} \mid w \geq x\}$; o procedimento de ordenação é chamado recursivamente para S_1 e S_2
 - $\mathcal{L}$ recebe a concatenação de S_1 , seguido de x , seguido de S_2

Quicksort: Pontos Decisivos

- Dois pontos decisivos para bom desempenho:
 1. Escolha do pivô;
 2. Particionamento da tabela.

Quicksort: Escolha do Pivô

- Estratégia comum é selecionar primeiro elemento como **pivô**
 - Se tabela ordenada na ordem inversa à desejada, provoca o pior desempenho

0	1	2	3	4	5	6
95	60	51	42	40	39	37

primeiro elemento

Quicksort: Escolha do Pivô 2

- Estratégia alternativa é a escolha aleatória do **pivô**
 - Mas a geração de números aleatórios poderia pesar no tempo de execução

0	1	2	3	4	5	6
40	37	95	42	39	51	60

aleatório

Quicksort: Escolha do Pivô 3

- Idealmente escolher **pivô** tal que a partição da tabela em dois subconjuntos de dimensão equivalente
 - A mediana da tabela deveria ser selecionada ..
Infelizmente, o cálculo custoso

0	1	2	3	4	5	6
40	37	95	42	39	51	60

mediana

Quicksort: Escolha do Pivô 4

- Uma solução utilizada com bons resultados é a escolha da mediana dentre três elementos: o primeiro, o último e o central

0	1	2	3	4	5	6
40	37	95	42	39	51	60

mediana

Quicksort: Particionamento

- **Pivô** é afastado da tabela a ser percorrida
 - Coloque-o na última posição e considere somente o restante da tabela
- Em seguida, dois ponteiros são utilizados:
 - i apontando para primeiro elemento
 - Percorra enquanto valores apontados são menores do que o **pivô**
 - j inicializado na penúltima posição
 - Percorra enquanto valores apontados são maiores do que o **pivô**

Quicksort: Particionamento (Cont.)

- Percurso interrompido quando elemento $i > \text{pivô}$ e elemento $j < \text{pivô}$
 - Se $i < j$, troca-se os elementos e continue
 - Se $i > j$, partição está determinada
- **Pivô**, que se encontra na última posição da tabela, deve ser trocado com o elemento de índice i
- Após troca, elementos $< i$ são menores do que o **pivô**; o índice i indica a posição definitiva do **pivô**, e os elementos $> i$ formam o conjunto de elementos maiores do que o **pivô**

Quicksort: Exemplo

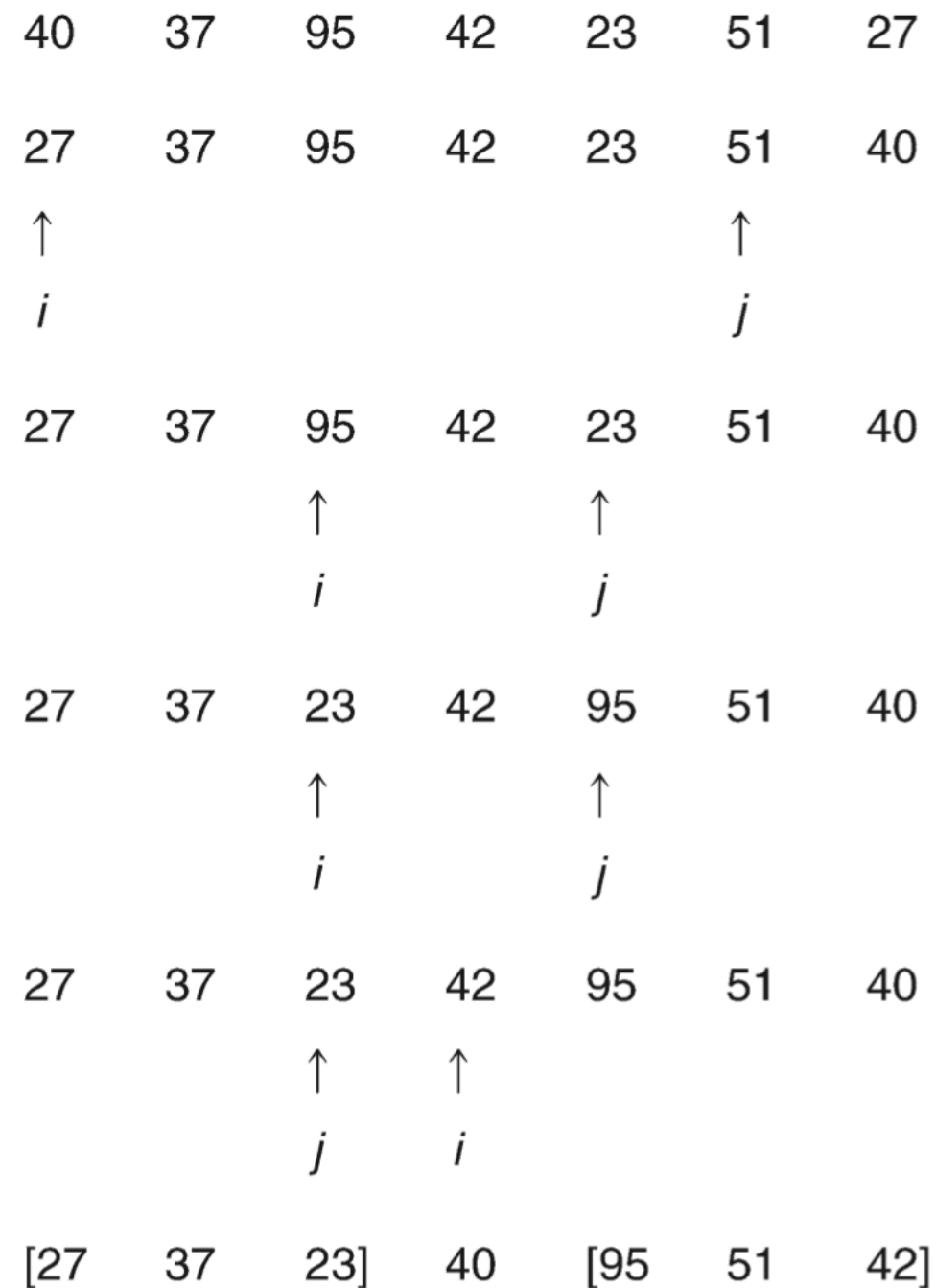


FIGURA 7.4 Ordenação rápida.

Quicksort: Algoritmo

```
1. void quicksort(int v[], int ini, int fim) {
2.     if (fim - ini < 2) {
3.         if (fim - ini == 1) {
4.             if (v[ini] > v[fim])
5.                 troque(&v[ini], &v[fim]);
6.         }
7.     } else {
8.         int mediana = pivo(v, ini, fim);
9.         troque(&v[mediana], &v[fim]);
10.
11.         int i = ini, j = fim - 1;
12.         int chave = v[fim];
13.
14.         while (j >= i) {
15.             while (v[i] < chave) i++;
16.             while (v[j] > chave) j--;
17.             if (j >= i) {
18.                 troque(&v[i], &v[j]);
19.             }
20.         }
21.     }
22. }
```

```
1. void quicksort(int v[], int ini, int fim) {
2.     if (fim - ini < 2) {
3.         if (fim - ini == 1) {
4.             if (v[ini] > v[fim])
5.                 troque(&v[ini], &v[fim]);
6.         }
7.     } else {
8.         int mediana = pivo(v, ini, fim);
9.         troque(&v[mediana], &v[fim]);
10.
11.         int i = ini, j = fim - 1;
12.         int chave = v[fim];
13.
14.         while (j >= i) {
15.             while (v[i] < chave) i++;
16.             while (v[j] > chave) j--;
17.             if (j >= i) {
18.                 troque(&v[i], &v[j]);
19.                 i++; j--;
20.             }
21.         }
22.         troque(&v[i], &v[fim]);
23.         quicksort(v, ini, i - 1);
24.         quicksort(v, i + 1, fim);
25.     }
26. }
```

Quicksort: Complexidade

- $O(n \log n)$ no **melhor caso** e no **caso médio**
- Embora o **pior caso** não é bom $O(n^2)$, o seu **caso médio** é muito melhor que o **pior caso**
 - Então, *quicksort* é um dos melhores algoritmos de ordenação utilizando comparação entre os elementos

Resumo Complexidades

Algoritmo	Melhor	Médio	Pior	Memória
Ordenação por Bolha	$O(n^2)^*$	$O(n^2)$	$O(n^2)$	1
Ordenação por Inserção	$O(n)$	$O(n^2)$	$O(n^2)$	1
Ordenação por Intercalação	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)^*$

Referências

- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010
- ZIVIANI, Nivio. Projeto de Algoritmos com Implementações em Pascal e C (3a. edição). Editora Cengage Learning, 2010.